

Le module math et les fonctions

1 La notion de fonction

`def nom_fonction(paramètres)` : définit une nouvelle fonction, les deux points entraînent une indentation délimitant la déclaration de la fonction. Le bloc peut servir à effectuer une série d'actions, mais le plus souvent il se termine par `return` pour renvoyer une ou plusieurs valeurs.

Exemples :

<pre>def f(n): return 2*n+1</pre>	La fonction ci-contre renvoie l'image de la variable n par la fonction $f: n \mapsto 2n+1$ On l'appelle dans la console Python en notant : <pre>>>> f(5) 11</pre>
<pre>def g(a,b): return -b/(2*a)</pre>	La fonction ci-contre a deux paramètres a et b. On l'appelle en indiquant ses 2 paramètres : <pre>>>> g(5,2) -0.2</pre>
<pre>def phrase(c): return c*5</pre>	La fonction <code>phrase</code> ci-contre admet une chaîne de caractère en paramètre. <pre>>>> phrase("Ho") 'HoHoHoHoHo'</pre>

Les docstrings sont des chaînes de caractères qui doivent être placées juste en dessous des définitions de fonction et entre 3 apostrophes `'''`.

On définit ainsi les variables de la fonction en entrée (IN) et celles en sortie de la fonction (OUT).

Les docstrings sont récupérables dynamiquement avec la fonction `help()`.

Exemple :

```
def f(x):  
    ''' In : x décimal (un flottant)  
    Out : image de x par f définie par f(x) = x2 + 1'''  
    return x * x + 1
```

Remarque : On peut aussi définir une fonction sans paramètre en ne mettant rien entre les parenthèses.

```
def SansParametre():
```

Pour chaque fonction, écrire une docstring

Exercice1 :

Écrire une fonction « solde » qui prend en paramètre un prix et le pourcentage de réduction et renvoie le prix soldé.

Exercice2

Écrire une fonction « salut » qui prend en paramètre une variable `prenom` et renvoie une chaîne de caractères constituée de 20 fois la phrase « Bonjour `prenom` »

Exercice3

Écrire une fonction de paramètre `r` qui renvoie le périmètre d'un cercle de rayon `r` puis une autre (aussi de paramètre `r`) qui renvoie l'aire du disque de rayon `r`.

2 Le module math

Les fonction mathématiques de base sont présentes dans le module math qu'il faut appeler au début du programme sous la forme `import math`. Pour le nombre π écrire tout simplement : `math.pi`

Astuce : En écrivant `from math import *` au début on importe directement toutes les fonctions et `from math import pi` on importe le nombre pi (enfin une valeur approchée!) ce qui permet d'écrire directement pi (au lieu de math.pi)

Les fonctions mathématiques sont accessibles en tapant `help(math)` dans la console.

Les fonctions mathématiques de bases :

Opérations	Interprétation	Exemples de syntaxe	Remarque
<code>+, -, *, /</code>	addition, soustraction, multiplication et division		
<code>a//b</code>	Partie entière de la division de a par b	<pre>> 12//11 1</pre>	$12 \div 11 \approx 1,0909$
<code>a % b</code>	Reste de la division euclidienne de a par b	<pre>> 17 % 3 2</pre>	$17 = 3 \times 5 + 2$
<code>int(a)</code>	partie entière	<pre>> int(12.123) 12</pre>	
<code>divmod(a,b)</code>	Quotient et Reste de la division euclidienne de a par b	<pre>> divmod(20,3) (6,2)</pre>	$20 = 3 \times 6 + 2$
<code>a**b</code> ou <code>pow(a,b)</code>	a Puissance b	<pre>> 2**3 ou pow(2,3) 8</pre>	$2^3 = 8$
<code>a**(1/2)</code>	Racine carrée $\sqrt{a}$	<pre>> 9**(1/2) 3</pre>	$\sqrt{9} = 3$
<code>a**(1/n)</code>	Racine $n^{\text{ième}}$ de a : $\sqrt[n]{a}$	<pre>> 27**(1/3) 3</pre>	$\sqrt[3]{27} = 3$
<code>abs(x)</code>	Valeur absolue de x : $ x $	<pre>> abs(-5.2) 5.2</pre>	$ -5.2 = 5.2$
<code>round(a,n)</code>	Arrondie de a à 10^{-n} près	<pre>> round(2.2563,2) 2.26</pre>	

Exercice 4 :

On partage une corde en morceaux de 50cm

Écrire un algorithme avec une fonction de paramètre L la longueur de la corde et renvoie le nombre de morceaux obtenus et la longueur de corde restante.

Exercice 5 : fonctions et pourcentages

1 Écrire un algorithme avec une fonction de paramètres p et t qui calcul le prix final, après une évolution de t% d'un prix initial de p euros.

2 Modifier le programme pour obtenir un arrondi au centième du résultat.

Exercice 6 :

- 1 Écrire un algorithme avec une fonction qui prend en paramètre un nombre à 4 chiffres et qui renvoie le chiffre des milliers.
- 2 Même question avec une fonction qui renvoie le chiffre des dizaines

3 Affichage des résultats

Pour afficher les variables de différentes natures (str, int, float) on peut utiliser deux méthodes :

```
# Méthode 1
# On place le texte entre "" et les variables séparées par des virgules

var1 = 2
var2 = -9
print("Méthode 1 : var1 =", var1, " et var2 =", var2)

# Méthode 2 : f (format) devant une chaîne de caractères

print(f"Méthode 2 : var1 = {var1} et var2 = {var2}")
```

La méthode avec format est plus simple d'utilisation puisqu'il suffit de mettre le nom des variables à afficher entre accolades.

Pour afficher des variables de différentes natures en faisant varier la forme des variables numérique, on utilisera un affichage par formatage :

```
var3=775776878.68767

# Méthode 3 : Affichage par formatage
# %e pour forme a10^n
# %0.nf à 10^-n près

print("Méthode 3: var3 = %e, var3 = %0.2f " % (var3,var3))
```

```
# Méthode 3 : Affichage par formatage
# %d pour les entiers int(),
# %f pour les flottants float()
# %s pour les chaînes de caractères str()

var1=2      # var1 est un entier int()
var2=2.6    # var2 est un float()
var3="BEBE" # var3 est un str()

print("Méthode 3: var1 = %d, var2 =%f et var3 =%s" % (var1,var2,var3))
```

Exercice 7 :

Tester les méthodes précédentes dans la console python.

Exercice 8 :

Créer un algorithme avec une fonction sans paramètre demandant le nom de l'utilisateur, son age, et le résultat de 1/8, puis renvoie une chaîne de caractère disant bonjour, mentionnant le nom et l'age de l'utilisateur et sa réponse au calcul à 10^{-2} près.

Exemple :

Bonjour Blaise, vous avez 25 ans et votre reponse est 1,12.

Vérifier votre fonction en affichant ce qu'elle renvoie.